

Cy Critical Runway

Modélisation et vérification d'un système d'atterrissage critique

Rapport de projet en Akka Typed, Scala, réseau de Pétri et LTL

Version finale étudiante, claire
et fidèle au projet implémenté

Établissement : CY Tech

Auteurs : Fares Ben Mabrouk
Maxime CLEMENT
Mehdi Boulaich
Nassym Ben Abdallah
Ismail Aitlaalim

Date : 12 avril 2026

Résumé. Ce projet propose la conception d'un système distribué critique de gestion d'atterrissage à l'aide d'Akka Typed et Scala. Le système est modélisé autour de trois acteurs principaux — *Avion*, *Tour de contrôle* et *Piste* — puis abstrait au moyen d'un réseau de Pétri afin d'explorer son espace d'états et de vérifier plusieurs propriétés de sûreté et de vivacité. Le rapport présente l'architecture retenue, les choix d'implémentation, la formalisation LTL des garanties attendues, ainsi que la stratégie de validation par tests et analyse formelle.

Table des matières

1	Introduction	1
2	Objectifs du projet	1
3	Architecture du système Akka / Scala	2
3.1	Vue générale	2
3.2	États métier	2
3.2.1	États d'un avion	2
3.2.2	États de la piste	2
3.3	Messages principaux	3
3.4	Gestion de la concurrence	3
3.5	Supervision	4
4	Traduction en réseau de Pétri	4
4.1	Principe de modélisation	4
4.2	Correspondance entre le code et les transitions	5
4.3	Intérêt du modèle formel	5
5	Propriétés structurelles et invariants métier	5
5.1	Exclusion mutuelle sur la piste	5
5.2	Unicité de l'état de la piste	5
5.3	Conservation du nombre d'avions	6
5.4	Pas d'autorisation sans réservation	6
5.5	Progression correcte vers le sol	6
6	Formalisation LTL	6
6.1	Pourquoi utiliser la LTL	6
6.2	Propositions atomiques	6
6.3	Propriétés de sûreté	7
6.4	Propriétés de vivacité	7
6.5	Positionnement méthodologique	8
7	Implémentation des tests	8
7.1	Objectif des tests	8

7.2	Jeu de tests réalisé	8
7.3	Commentaires sur la validation	9
8	Analyseur formel et résultats	9
9	Limites du projet	9
10	Bilan critique	10
11	Conclusion	10
	Bibliographie minimale	11

1 Introduction

Les systèmes distribués critiques apparaissent dans de nombreux domaines : transport, finance, télécommunications ou encore industrie. Dans ce type de systèmes, une erreur de coordination peut produire un comportement dangereux ou incohérent. Le sujet de ce projet demande de modéliser puis de vérifier un système critique distribué en s'appuyant sur Akka, Scala, les réseaux de Pétri et une formalisation en logique temporelle LTL.

Nous avons choisi de travailler sur un scénario d'atterrissage aéroportuaire. Ce choix présente plusieurs avantages :

- la **ressource critique** est très claire : la piste ;
- les **interactions concurrentes** sont naturelles : plusieurs avions peuvent demander l'atterrissage en même temps ;
- les **propriétés métier** sont faciles à exprimer : pas de double occupation de piste, pas d'atterrissage sans autorisation, progression correcte jusqu'au sol ;
- le **lien entre simulation et vérification formelle** est simple à établir.

L'objectif n'était pas de construire une simulation aéroportuaire exhaustive, mais de réaliser un projet **cohérent, maîtrisé et vérifiable**, avec un niveau de complexité compatible avec un projet étudiant. La stratégie retenue a donc été de privilégier :

- une architecture simple mais rigoureuse ;
- des invariants clairement identifiés ;
- une implémentation Akka Typed propre ;
- un modèle de réseau de Pétri aligné avec le comportement du code ;
- une validation à la fois par **tests** et par **analyse d'espace d'états**.

2 Objectifs du projet

Le projet doit répondre à quatre objectifs techniques principaux :

1. **Modéliser** un système distribué critique à l'aide d'acteurs Akka ;
2. **Identifier** les interactions concurrentes et les états sensibles ;
3. **Traduire** ce fonctionnement en un modèle formel de réseau de Pétri ;
4. **Vérifier** des propriétés structurelles, métier et temporelles.

Dans notre cas, cela revient à garantir au minimum les points suivants :

- un seul avion peut utiliser la piste à un instant donné ;
- un avion ne peut pas atterrir sans autorisation ;
- les demandes concurrentes sont traitées sans incohérence ;

- les avions finissent par atteindre l'état *Au sol* dans le scénario nominal ;
- le modèle formel ne présente pas de deadlock non terminal.

3 Architecture du système Akka / Scala

3.1 Vue générale

Le système repose sur trois acteurs principaux :

- **Avion** : représente un avion individuel et son cycle de vie ;
- **TourDeContrôle** : centralise les demandes, gère la file d'attente FIFO et coordonne l'accès à la piste ;
- **Piste** : représente la ressource critique et impose une exclusion mutuelle stricte.

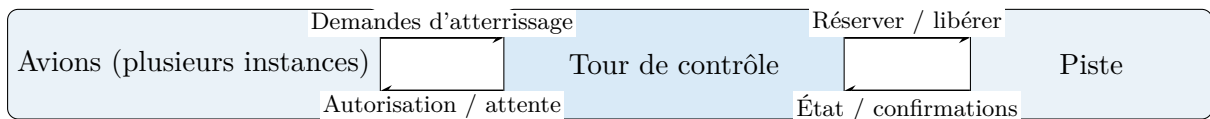


FIGURE 1 – Architecture logique simplifiée du système distribué.

Cette architecture nous a semblé être le bon compromis entre simplicité et richesse : elle permet de représenter de vraies interactions concurrentes tout en restant assez compacte pour être modélisée formellement.

3.2 États métier

3.2.1 États d'un avion

Chaque avion suit le cycle de vie suivant :

- `AvionEnVol`
- `AvionEnAttente`
- `AvionAutorise`
- `AvionAtterrissageEnCours`
- `AvionAuSol`

Ce découpage est volontairement simple. Il permet de représenter le comportement du système sans multiplier artificiellement les états.

3.2.2 États de la piste

La piste possède trois états exclusifs :

- `PisteLibre`

- `PisteReservee`
- `PisteOccupee`

L'introduction de l'état *Réservée* est importante : elle distingue le moment où la piste est attribuée à un avion du moment où elle est effectivement occupée physiquement.

3.3 Messages principaux

TABLE 1 – Messages principaux du protocole.

Message	Émetteur principal	Rôle
<code>DemandeAtterrissage</code>	Avion	Demande d'accès au système d'atterrissage
<code>AutorisationAccordee</code>	Tour	Autorisation d'atterrir pour un avion donné
<code>MiseEnAttente</code>	Tour	Placement d'un avion dans la file d'attente
<code>ReserverPour</code>	Tour	Demande de réservation de piste
<code>ReservationAccordee</code>	Piste	Confirmation qu'une réservation a été acceptée
<code>Occuper</code>	Avion	Occupation effective de la piste
<code>FinAtterrissage</code>	Avion	Notification de fin de manœuvre à la tour
<code>Liberer</code>	Tour	Demande de libération de la piste
<code>LiberationConfirme</code>	Piste	Confirmation que la piste est redevenue libre
<code>EtatPisteActuel</code>	Piste	Réponse de synchronisation à la tour

3.4 Gestion de la concurrence

La concurrence est gérée par deux mécanismes complémentaires :

- la **piste** impose une exclusion mutuelle stricte ;
- la **tour de contrôle** maintient une **file FIFO** lorsque la piste n'est pas disponible.

Lorsqu'un avion demande l'atterrissage :

1. si la piste est libre, la tour demande une réservation ;
2. une fois la réservation confirmée, l'avion reçoit l'autorisation ;
3. l'avion occupe la piste et réalise sa manœuvre ;
4. à la fin, la piste est libérée ;
5. si des avions sont en attente, le premier de la file est traité.

3.5 Supervision

Le projet inclut une supervision simple : la tour de contrôle est redémarrée en cas de crash simulé. Après redémarrage, elle se resynchronise avec l'état courant de la piste. Cette supervision illustre un comportement de reprise réaliste, mais il faut souligner qu'il ne s'agit pas d'une persistance complète de l'état métier.

Autrement dit, la reprise mise en œuvre ici est une **resynchronisation partielle**, pas une tolérance aux pannes exhaustive.

4 Traduction en réseau de Pétri

4.1 Principe de modélisation

Le réseau de Pétri reprend les états essentiels du système afin de représenter, de manière abstraite, les évolutions possibles du scénario d'atterrissage.

Les places principales du modèle sont les suivantes :

- EnVol
- EnAttente
- Autorise
- AtterrissageEnCours
- AuSol
- DansFile
- PisteLibre
- PisteReservee
- PisteOccupee

Ce modèle est volontairement **agrégé** : il ne distingue pas chaque avion individuellement dans le marquage, mais il conserve le nombre total d'avions présents dans chaque état. Ce choix permet une exploration exhaustive plus simple tout en restant fidèle au comportement global du système Akka.

4.2 Correspondance entre le code et les transitions

TABLE 2 – Correspondance entre transitions du réseau de Pétri et comportement Akka.

Transition	Effet sur le marquage	Signification dans le code
T1	EnVol $\rightarrow$ EnAttente	émission de DemandeAtterrissage
T2	EnAttente + PisteLibre $\rightarrow$ Autorise + PisteReservee	réservation confirmée puis autorisation envoyée
T3	EnAttente $\rightarrow$ DansFile	avion placé en attente lorsque la piste n'est pas libre
T4	Autorise + PisteReservee $\rightarrow$ AtterrissageEnCours + PisteOccupee	occupation effective de la piste
T5	AtterrissageEnCours $\rightarrow$ AuSol	fin de manœuvre de l'avion
T6	PisteOccupee $\rightarrow$ PisteLibre	libération de la piste
T7	DansFile + PisteLibre $\rightarrow$ Autorise + PisteReservee	traitement FIFO du prochain avion

4.3 Intérêt du modèle formel

Le réseau de Pétri nous permet de raisonner sur :

- les **séquences possibles** d'exécution ;
- la **cohérence structurelle** des états ;
- l'**absence de deadlocks** dans le scénario nominal ;
- l'atteignabilité d'un **état terminal correct** ;
- la vérification d'invariants qui seraient plus difficiles à garantir uniquement par lecture du code.

5 Propriétés structurelles et invariants métier

Nous avons retenu les propriétés suivantes comme étant les plus importantes pour le système.

5.1 Exclusion mutuelle sur la piste

Deux avions ne doivent jamais occuper la piste en même temps. C'est la propriété de sûreté centrale du projet.

5.2 Unicité de l'état de la piste

La piste doit toujours être dans exactement un état parmi : *Libre*, *Réservée*, *Occupée*. Cela évite les situations incohérentes où la piste serait simultanément considérée comme libre et occupée.

5.3 Conservation du nombre d'avions

Le modèle ne doit ni perdre ni créer d'avions. À tout instant, la somme des avions présents dans les différents états doit rester constante.

5.4 Pas d'autorisation sans réservation

Un avion ne peut pas être dans l'état *Autorisé* si la piste n'a pas été préalablement réservée pour lui.

5.5 Progression correcte vers le sol

Dans le scénario nominal, un avion autorisé doit finir par atteindre l'état *Au sol*, et la piste doit ensuite être libérée.

6 Formalisation LTL

6.1 Pourquoi utiliser la LTL

La logique temporelle linéaire (LTL) permet de formaliser les garanties attendues d'un système concurrent. Dans ce projet, elle est utilisée comme **langage de spécification** : elle ne sert pas à implémenter un model checker complet, mais à écrire proprement les propriétés que notre système doit respecter.

Nous vérifions ensuite ces propriétés, ou leurs équivalents opérationnels, à travers :

- les invariants sur le modèle Akka ;
- les tests automatisés ;
- l'exploration de l'espace d'états du réseau de Pétri.

6.2 Propositions atomiques

On introduit les propositions atomiques suivantes :

- L : la piste est libre ;
- R : la piste est réservée ;
- O : la piste est occupée ;
- A : au moins un avion est autorisé ;
- C : au moins un avion est en cours d'atterrissage ;
- T : tous les avions sont au sol.

Pour un avion donné i , on note aussi :

- Dem_i : l'avion i a émis une demande ;
- $Wait_i$: l'avion i est en attente ;
- $Auth_i$: l'avion i est autorisé ;
- $Ground_i$: l'avion i est au sol.

6.3 Propriétés de sûreté

S1 – la piste ne peut pas être dans deux états à la fois.

$$G\neg(L \wedge R) \wedge G\neg(L \wedge O) \wedge G\neg(R \wedge O)$$

Cette propriété exprime l'unicité de l'état de la piste.

S2 – une autorisation implique une réservation préalable.

$$G(A \rightarrow R)$$

Cette formule traduit le fait qu'un avion ne peut être autorisé à atterrir que si la piste a été réservée.

S3 – un atterrissage en cours implique une piste occupée.

$$G(C \rightarrow O)$$

Cette propriété empêche toute incohérence entre l'état avion et l'état de la ressource critique.

6.4 Propriétés de vivacité

V1 – toute occupation de piste finit par se terminer.

$$G(O \rightarrow FL)$$

Autrement dit, une piste occupée doit redevenir libre à terme.

V2 – tout scénario nominal finit par atteindre un état terminal correct.

$$FT$$

Cette propriété formalise le fait qu'à terme, tous les avions du scénario atteignent l'état *Au sol*.

V3 – toute demande reçoit finalement une réponse.

$$G(Dem_i \rightarrow F(Wait_i \vee Auth_i))$$

Un avion qui demande l'atterrissage ne doit pas rester sans retour du système.

V4 – tout avion autorisé finit par atteindre le sol.

$$G(Auth_i \rightarrow FGround_i)$$

6.5 Positionnement méthodologique

Il est important de préciser que notre travail n'implémente pas un **model checker LTL générique**. Nous avons adopté une démarche plus légère et adaptée au projet :

1. formalisation des propriétés en LTL ;
2. abstraction du système en réseau de Pétri ;
3. exploration exhaustive de l'espace d'états ;
4. contrôle des invariants, des deadlocks et de l'atteignabilité des états terminaux.

Cette démarche reste cohérente avec le cadre du sujet et suffisante pour démontrer la rigueur de la modélisation choisie.

7 Implémentation des tests

7.1 Objectif des tests

Les tests servent à valider le comportement concret du système Akka. Ils complètent la partie formelle en montrant que l'implémentation suit bien les propriétés attendues.

7.2 Jeu de tests réalisé

Dans la version finale du projet, nous avons mis en place **8 tests** :

N°	Test	Intérêt
1	Autorisation immédiate si piste libre	Vérifie le scénario nominal le plus simple
2	Mise en attente d'un deuxième avion si piste occupée	Vérifie la gestion d'une demande concurrente
3	Respect de l'ordre FIFO	Vérifie la cohérence de la file d'attente
4	Ignorer une libération avec un mauvais identifiant	Vérifie la robustesse de la ressource critique
5	Retour à l'état libre quand la file est vide	Vérifie la libération correcte de la piste
6	Resynchronisation après crash de la tour	Vérifie la supervision simple et la reprise partielle

7	Scénario réel avec deux avions	Vérifie le cycle complet avec de vrais acteurs Avion
8	Scénario réel avec trois avions	Vérifie la progression logique du système sur un cas plus riche

7.3 Commentaires sur la validation

L'intérêt du jeu de tests est double :

- vérifier les **règles métier locales** ;
- valider le **fonctionnement bout en bout** du système concurrent.

Le choix d'inclure des tests avec de vrais avions est particulièrement important, car il permet de vérifier non seulement la logique de la tour et de la piste, mais aussi l'enchaînement réel des messages dans un scénario complet.

8 Analyseur formel et résultats

L'analyseur du réseau de Pétri effectue une exploration en largeur (BFS) de l'espace d'états à partir du marquage initial.

Dans notre modèle :

- le nombre d'avions est fixé à 3 ;
- chaque transition modifie le marquage conformément au protocole Akka ;
- les invariants sont vérifiés sur chaque état exploré ;
- les états sans successeur sont analysés pour détecter un éventuel deadlock non terminal.

Cette analyse permet de confirmer les points suivants dans le scénario nominal :

- aucune violation des invariants définis n'est détectée ;
- aucun deadlock non terminal n'apparaît ;
- un état terminal où tous les avions sont au sol est atteignable ;
- la piste est bien libérée à la fin des scénarios explorés.

9 Limites du projet

Même si le projet est cohérent et fonctionnel, certaines limites doivent être assumées avec honnêteté.

- Le système ne gère qu'une **seule piste**. Cela simplifie le modèle mais réduit le réalisme.
- La **supervision** est volontairement limitée : après crash, la tour se resynchronise avec la piste mais ne restaure pas toute son histoire métier.

- Le réseau de Pétri est un **modèle agrégé** : il capture la dynamique globale sans distinguer chaque avion par un identifiant propre dans le marquage.
- La partie LTL correspond à une **spécification formelle des propriétés** ; elle n'est pas évaluée par un moteur LTL générique.

Ces limites ne remettent pas en cause la validité du projet, mais elles définissent clairement son périmètre.

10 Bilan critique

Le principal point fort du projet est la **cohérence entre les différentes couches** :

- architecture Akka claire ;
- machine à états métier simple mais pertinente ;
- ressource critique bien isolée ;
- file d'attente FIFO ;
- réseau de Pétri aligné avec le comportement du code ;
- propriétés de sûreté et de vivacité clairement explicitées.

Le projet remplit donc bien son rôle pédagogique : il montre comment passer d'une implémentation concurrente à une modélisation plus formelle, puis comment utiliser cette modélisation pour raisonner sur la correction du système.

11 Conclusion

Ce projet nous a permis de concevoir, implémenter et analyser un système distribué critique de gestion d'atterrissage. Le système Akka/Scala fournit une simulation concurrente crédible autour de trois acteurs principaux : les avions, la tour de contrôle et la piste. Le réseau de Pétri abstrait ce comportement afin d'en explorer les états possibles et d'en vérifier plusieurs propriétés fondamentales.

L'intérêt principal du travail réalisé réside dans l'articulation entre :

- la **modélisation logicielle** ;
- la **formalisation** des propriétés ;
- la **validation** par tests et exploration d'états.

Au final, le projet montre qu'il est possible de construire un système critique simple mais rigoureux, de le rendre vérifiable et d'en justifier le comportement sans tomber dans une complexité excessive.

Bibliographie minimale

- Documentation Akka Typed.
- T. Murata, *Petri Nets : Properties, Analysis and Applications*.
- M. Huth, M. Ryan, *Logic in Computer Science*.
- Cours et consignes du module.